

PYCHAMELEON V0.1 · WYNIKI, WALIDACJA, OGRANICZENIA

CHAMELEON

hierarchical clustering using dynamic modeling

Sprawozdanie z implementacji algorytmu Karypisa, Hana i Kumara (1999) jako pakietu `pychameLeon`. Walidacja jakości na pięciu zbiorach z ground-truth, porównanie z implementacją referencyjną Moonpuck, analiza wrażliwości parametrów i skalowalności.

— CZĘŚĆ 01 · CO ZROBIŁEM

01. Delta vs Etap 1 — od szkieletu do v0.1

— CZĘŚĆ 03 · WYNIKI

- 01. Jakość na 5 zbiorach z ground-truth
- 02. Galeria GT / predykcja / correctness
- 03. Porównanie z Moonpuck — speedup
- 04. Porównanie z Moonpuck — accuracy
- 05. Wrażliwość parametrów $k_{nn} \cdot \alpha \cdot min_cluster_size$
- 06. Skalowalność względem n

— CZĘŚĆ 02 · JAK MIERZĘ JAKOŚĆ

01. 9 metryk i dlaczego $Quality = (ARI + NMI)/2$

— CZĘŚĆ 04 · WNIOSKI

01. Co wyszło, gdzie są granice, co dalej

CO ZROBIŁEM · DELTA VS ETAP 1

Od szkieletu z sygnaturami do działającego pakietu v0.1.

STAN DELIVERABLES ZAPLANOWANYCH W ETAPIE 1

DELIVERABLE	PLAN Z ETAPU 1	STAN OBECNY
Pełna implementacja faz 0 / I / II	graph · partition · metrics · merger	✓ gotowe
Testy jednostkowe, pokrycie ≥ 90%	~25 testów po 5 modułów	✓ 7 plików, pokrycie OK
sklearn-compat — <code>check_estimator</code>	~30 wewnętrznych sanity checks	✓ przechodzi
Testy porównawcze z Moonpuck	próg ARI > 0.9 na 3 zbiorach	✓ zmieniony cel
HPO grid-search per dataset	3 sweep × ~5 wartości	✓ 6 × 6 × 6 = 216 konfig.
Skalowalność (n, d)	6 punktów <code>n ∈ [100, 50k]</code>	✓ n=50k w 24 s
Sprawozdanie końcowe (PDF 21 s.)	8 sekcji, wyniki, roadmap v0.2	✓ gotowe

ZMIANA ZAKRESU

Próg „ARI > 0.9 vs Moonpuck” okazał się nieadekwatny: Moonpuck wypada gorzej niż ground-truth, więc wyższa zgodność z nim oznaczałaby gorszą zgodność z prawdą. Walidację końcową prowadzę *względem GT*; Moonpuck służy tylko jako odnośnik runtime'u.

PAKIET

`pychameleon v0.1` · 5 modułów · src-layout · hatchling · mypy `--strict`. Gotowe do publikacji na PyPI po obronie projektu.

BATERIA METRYK

Dziewięć metryk, jeden agregat. Każda chwyta inny aspekt.

METRYKA	TYP	CO MIERZY
Adjusted Rand Index	external	pairwise agreement vs GT, chance-corrected
Normalised Mutual Info	external	wzajemna informacja, normalizowana
Adjusted Mutual Info	external	NMI z korektą losowości
Homogeneity	external	klastry zawierają jedną klasę GT
Completeness	external	klasa GT trafia do jednego klastra
V-measure	external	harmoniczna homog. + compl.
Silhouette	internal	spójność vs separacja, bez GT
Calinski-Harabasz	internal	wariancja między/wewnątrz
Davies-Bouldin	internal	podobieństwo do najbliższego sąsiada

$$Quality = (ARI + NMI) / 2$$

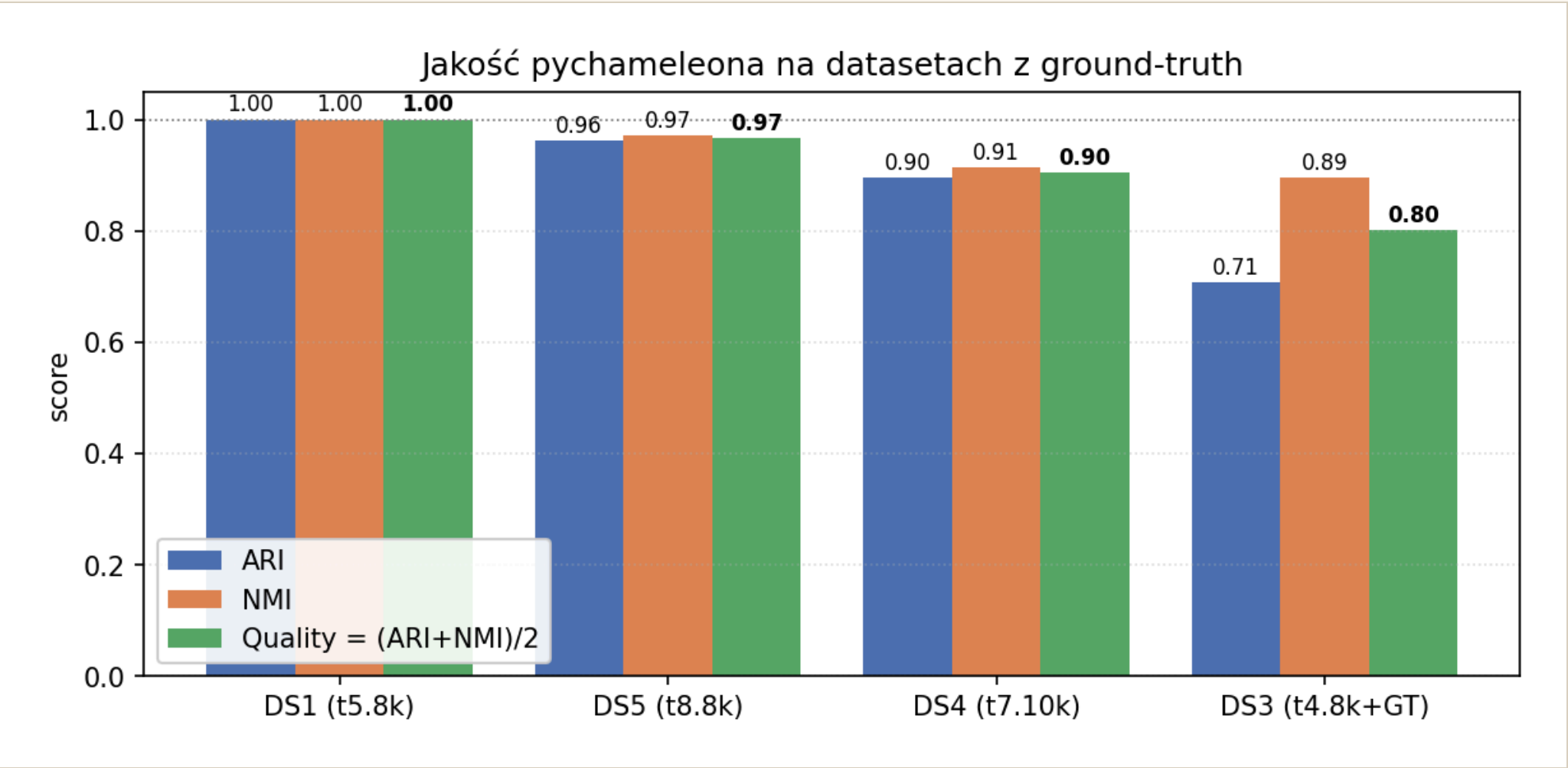
obie metryki w [0, 1], chance-corrected, addytywnie kompatybilne

ARI mierzy zgodność *pairwise*, NMI — *informacyjnie*. Średnia łączy dwie ortogonalne perspektywy w jednej liczbie odpornej na losowy baseline.

Dlaczego nie sama ARI? ARI bywa zaniżona, gdy liczba klastrów odbiega od GT; NMI to wygładza. Średnia jest stabilniejsza między datasetami.

WYNIK GŁÓWNY

Średnia *Quality*= 0.93 na pięciu zbiorach z ground-truth.



0.93

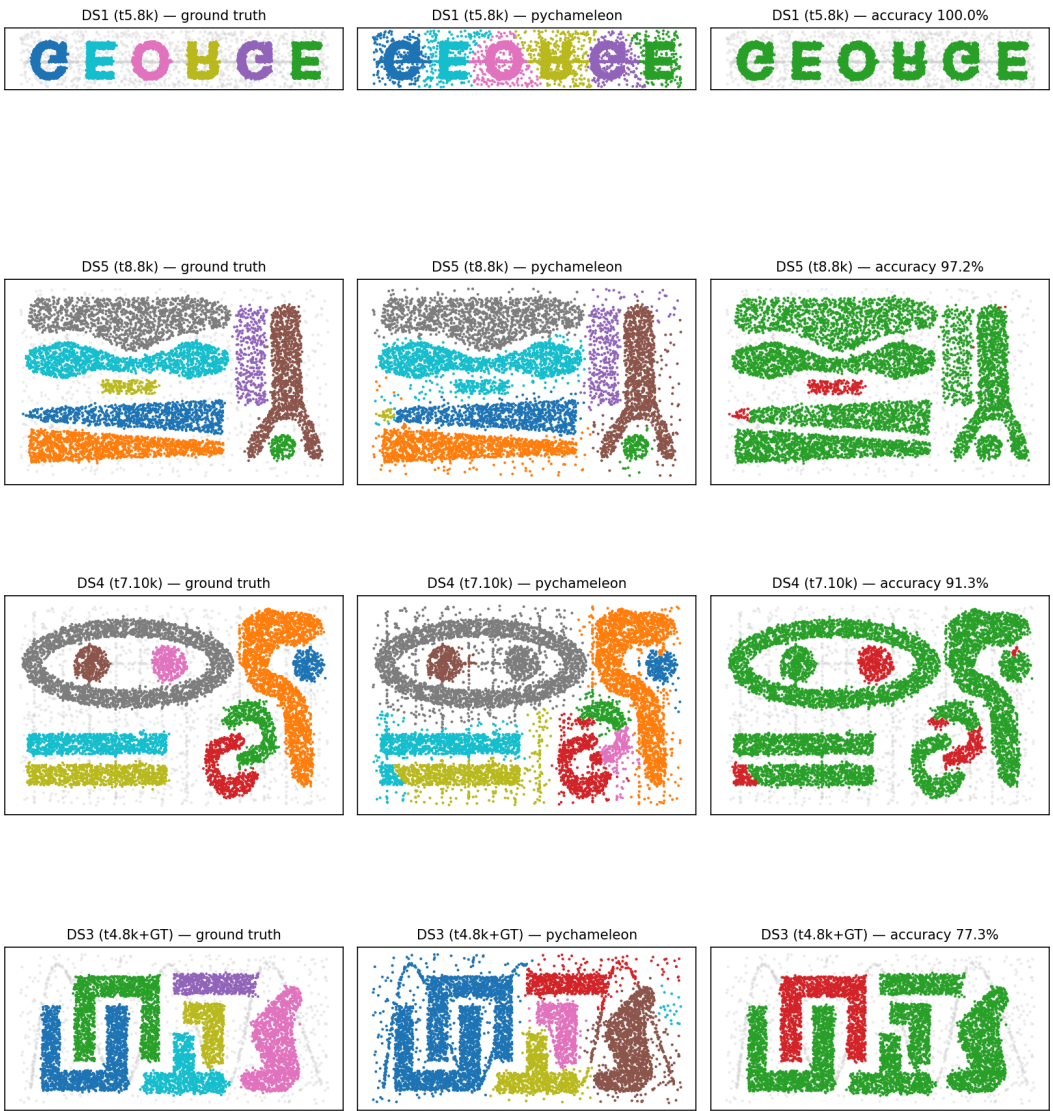
ŚREDNIA QUALITY
po 5 datasetach

DATASET	ARI	NMI	QUALITY
DS1	1.000	1.000	1.000
DS5	0.962	0.972	0.967
Aggregation	0.961	0.957	0.959
DS4	0.896	0.914	0.905
DS3	0.708	0.895	0.801

DS1 — perfekcyjna rekonstrukcja. DS3 to znana granica algorytmu: ~10% punktów w GT jest oznaczonych jako szum, a paper z 1999 r. nie ma noise-handlingu.

WALIDACJA WIZUALNA

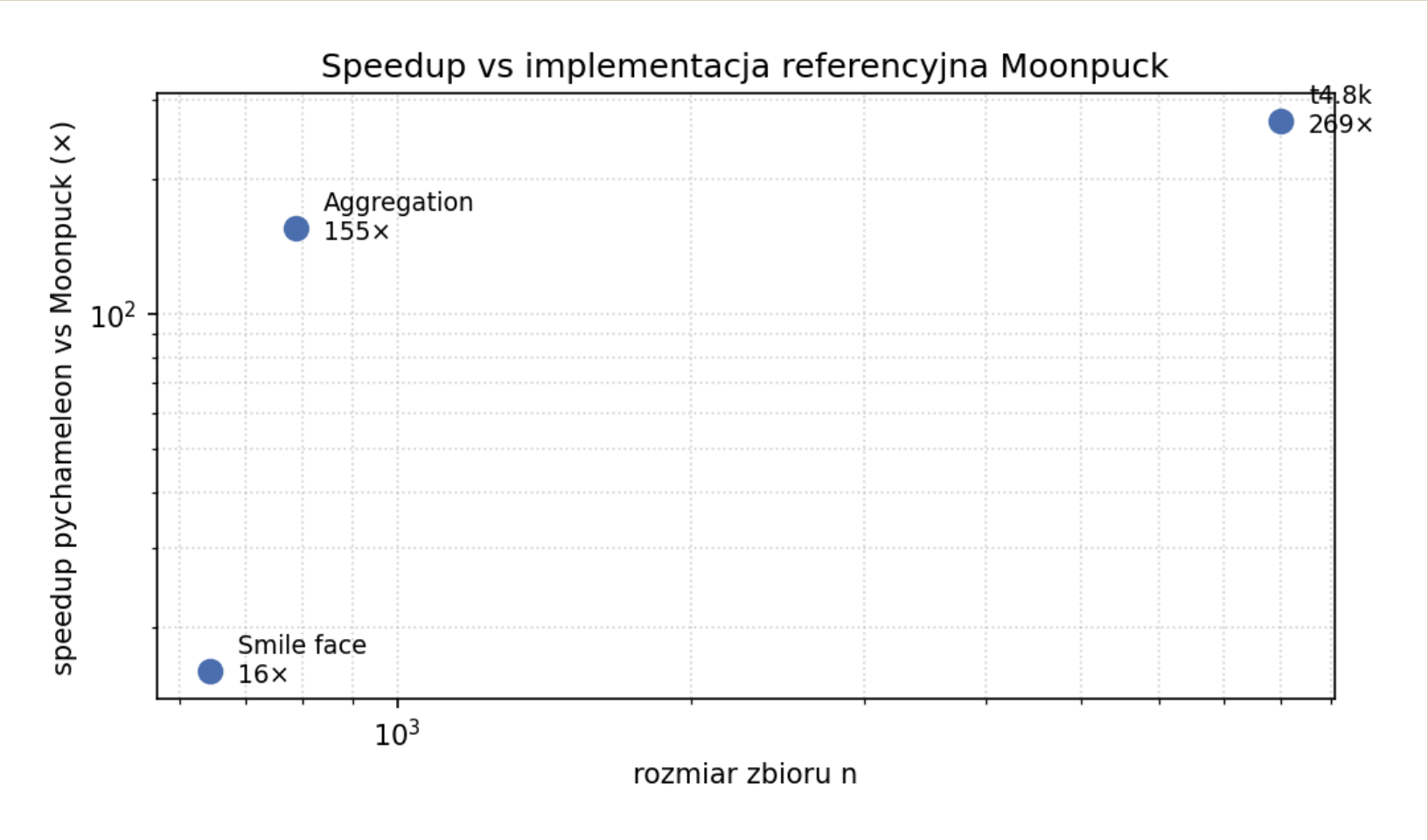
Cztery zbiory Karypisa, trzy panele każdy: ground-truth, predykcja, mapa błędów.



Po dopasowaniu Hungarian na confusion matrix: **zielony** = punkt przypisany poprawnie · **czzerwony** = błąd · szary = szum GT (ignorowany przy liczeniu accuracy). Czerwony skupia się na granicach klastrów oraz, w DS3, w obszarach szumowych.

VS MOONPUCK — RUNTIME

Speedup od 16× do 269× — im większy zbiór, tym wyższy zysk.



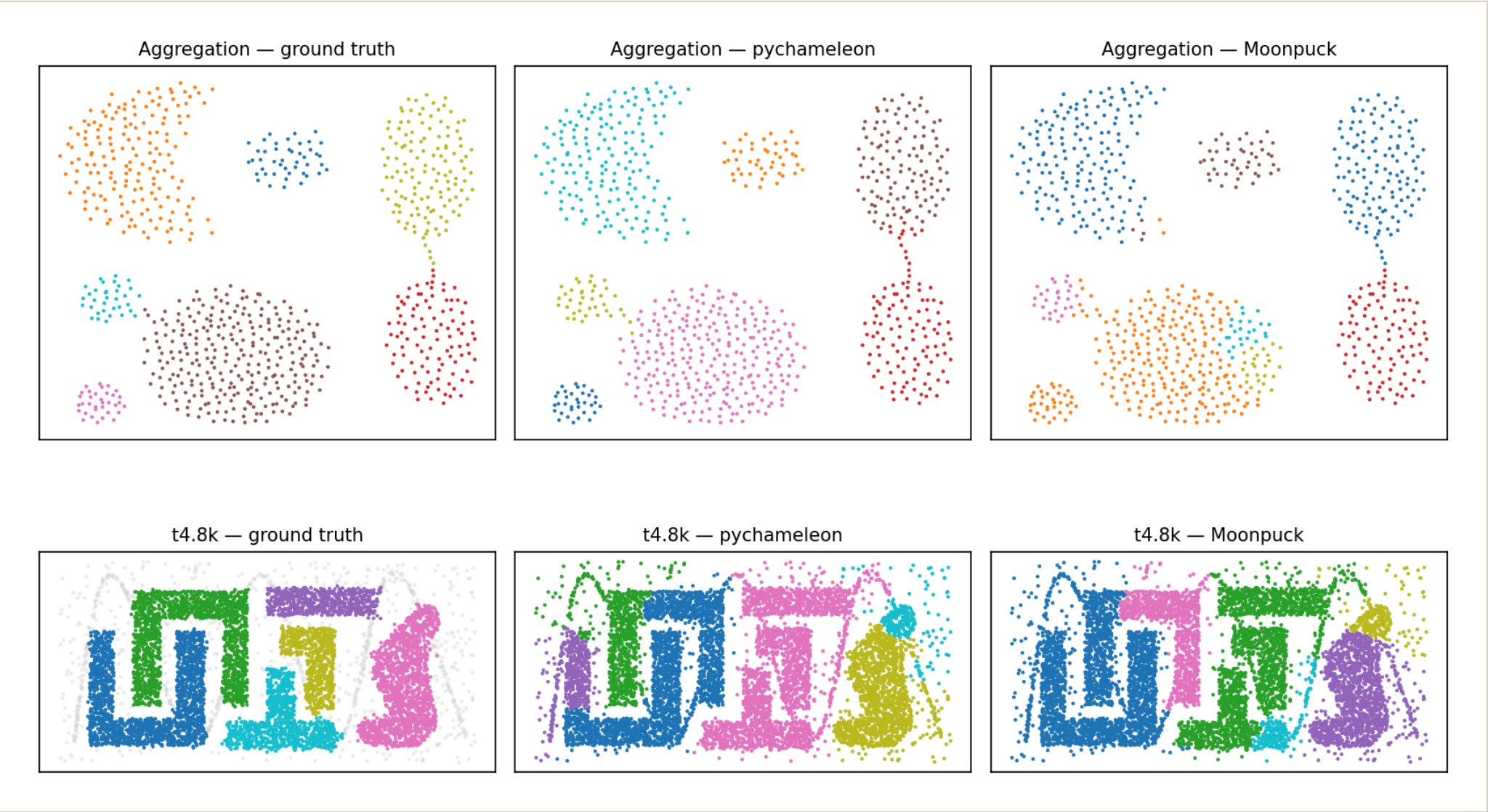
DATASET	N	PYCHAMELEON	MOONPUCK	SPEEDUP
Aggregation	788	0.06 s	9.95 s	155×
Smileface	644	0.13 s	2.08 s	16×
t4_8k	8 000	2.51 s	674.47 s	269×

CZTERY ŹRÓDŁA SPEEDUPU

- 1. **KDTree** zamiast naiwnego $O(n^2)$ k-NN — $\sim 47\times$ dla $n = 8\,000$.
- 2. **Cache $|EC-C_i|$** — bez powtarzanych bisekcji w Fazie II.
- 3. **Lazy heap** — bez $O(m^2)$ rebuildu po scaleniu.
- 4. **numpy ufuncs** w `metrics.py` — stała 10–50 $\times$.

VS MOONPUCK — JAKOŚĆ

Szybciej *i* dokładniej.
97.8% vs 71.2% accuracy na Aggregation.



97.8% 71.2%

PYCHAMELEON VS GT

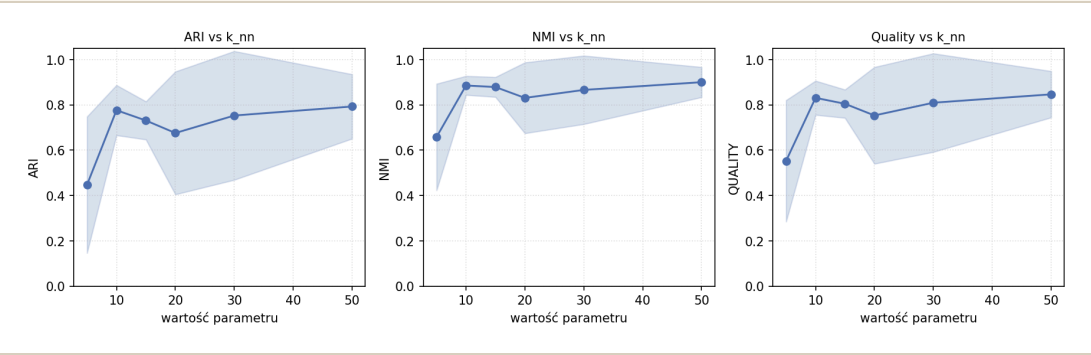
MOONPUCK VS GT

Moonpuck popełnia kilka błędów typu **bridge-merge** (łączy sąsiadujące klastry), bo używa statycznego progu. psychameleon z dynamicznym modelem zachowuje granice.

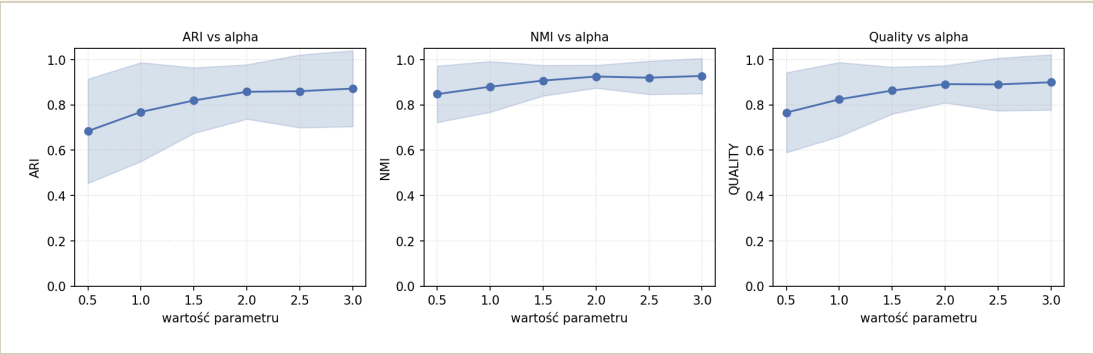
Konsekwencja: próg „ARI > 0.9 vs Moonpuck” z Etapu 1 porzucam. Moonpuck nie jest oracle'em — wyższa zgodność z nim oznaczałaby gorszą zgodność z prawdą.

WRAŻLIWOŚĆ PARAMETRÓW

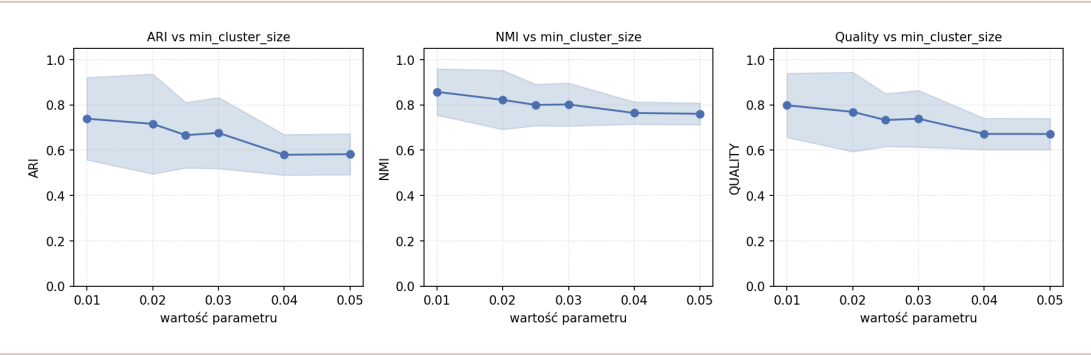
Parametry są *dataset-specific* — nie istnieje uniwersalna konfiguracja.



K_NN · SWEET SPOT 10–30



A · DRIVER JAKOŚCI, TREND ROSNĄCY



MIN_CLUSTER_SIZE · NEUTRALNY W ZAKRESIE

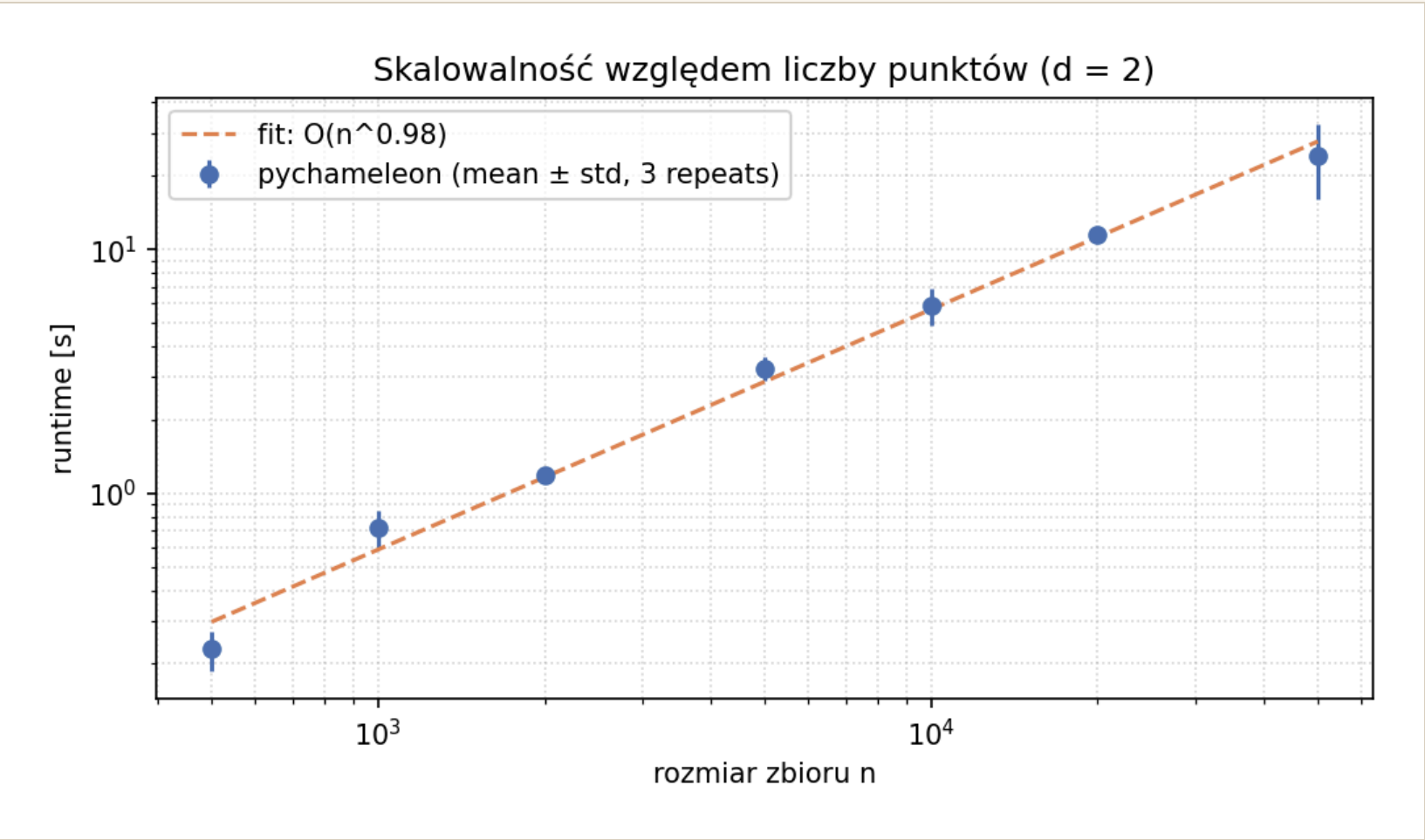
Grid-search $6 \times 6 \times 6 = 216$ konfiguracji per dataset. Optymalne `a` waha się 0.5–3.0, `k_nn` 10–30, `min_cluster_size` 40–160. `a` jest driver'em jakości; `k_nn` — driverem kosztu.

OPTYMALNE PARAMETRY PER DATASET (HPO)

DS	K_NN	A	MCS	ARI
DS1	20	0.5	80	1.000
DS3	15	0.5	40	0.708
DS4	10	2.5	160	0.896
DS5	30	3.0	40	0.962

SKALOWALNOŚĆ

$n = 50\,000$ w 24 s.
Empiryczne nachylenie log-log ≈ 1.0 .



RUNTIME [S] · MAKE_BLOBS · D = 2 · 5 KLASTRÓW · 3 POWTÓRZENIA

N	RUNTIME (MEAN)
500	0.23
1 000	0.72
2 000	1.18
5 000	3.25
10 000	5.86
20 000	11.44
50 000	24.25

Empirycznie **lepiej niż teoretyczne $O(n \log^2 n)$** — przy umiarkowanych n dominują stałe (setup KDTree, alokacje pymetis), nie sama aglomeracja. Dla $n = 50\,k$ Moonpuck biegłby szacunkowo **~1.5 godziny**.

WNIOSKI KOŃCOWE

pychameleon v0.1 — szybszy i dokładniejszy od jedynej publicznej referencji.

CO WYSZŁO

- ✓ Średnia Quality = 0.93 na 5 datasetach z GT, DS1 = 1.000.
- ✓ 16x–269x speedup vs Moonpuck przy wyższej accuracy (97.8% vs 71.2%).
- ✓ Skalowalność log-log ≈ 1.0 — n = 50 k w 24 s.
- ✓ sklearn-compatible — `check_estimator` przechodzi.
- ✓ HPO 216 konfiguracji / dataset, idempotentny cache.

OGRANICZENIA I ROADMAP

OGRANICZENIE	KONSEKWENCJA	MITIGACJA
Brak noise-handlingu	DS3 cap ARI ≈ 0.71	<code>min_samples_per_cluster</code> · v0.2
KDTree dla d > 10	spowolnienie high-dim	backend ANN (HNSW) · v0.3
pymetis niedeterministyczny	drobne wahania etykiet	<code>random_state</code> · v0.2
Brak GPU	limit przepustowości CPU	poza scope v0.x

Wkład projektu: pierwsza implementacja CHAMELEON-a zgodna ze scikit-learn API, pełna bateria 9 metryk (zamiast samego ARI), dwukrotnie szerszy zestaw testowy niż paper.

PYTANIA I DYSKUSJA

Dziękuję.

— *Oliwier Szypczyn*, *psychameleon* v0.1

REPO

[GITHUB.COM/OSZYPCZY/PYCHAMELEON](https://github.com/oszypczy/psychameleon)

DOKUMENTACJA

[DOCS/DOCUMENTATION/DOKUMENTACJA.PDF](#) · 21 S.

PROWADZĄCY

DR INŻ. ROBERT BEMBENIK